

Creating/Processing HTML Forms from Django

CS412

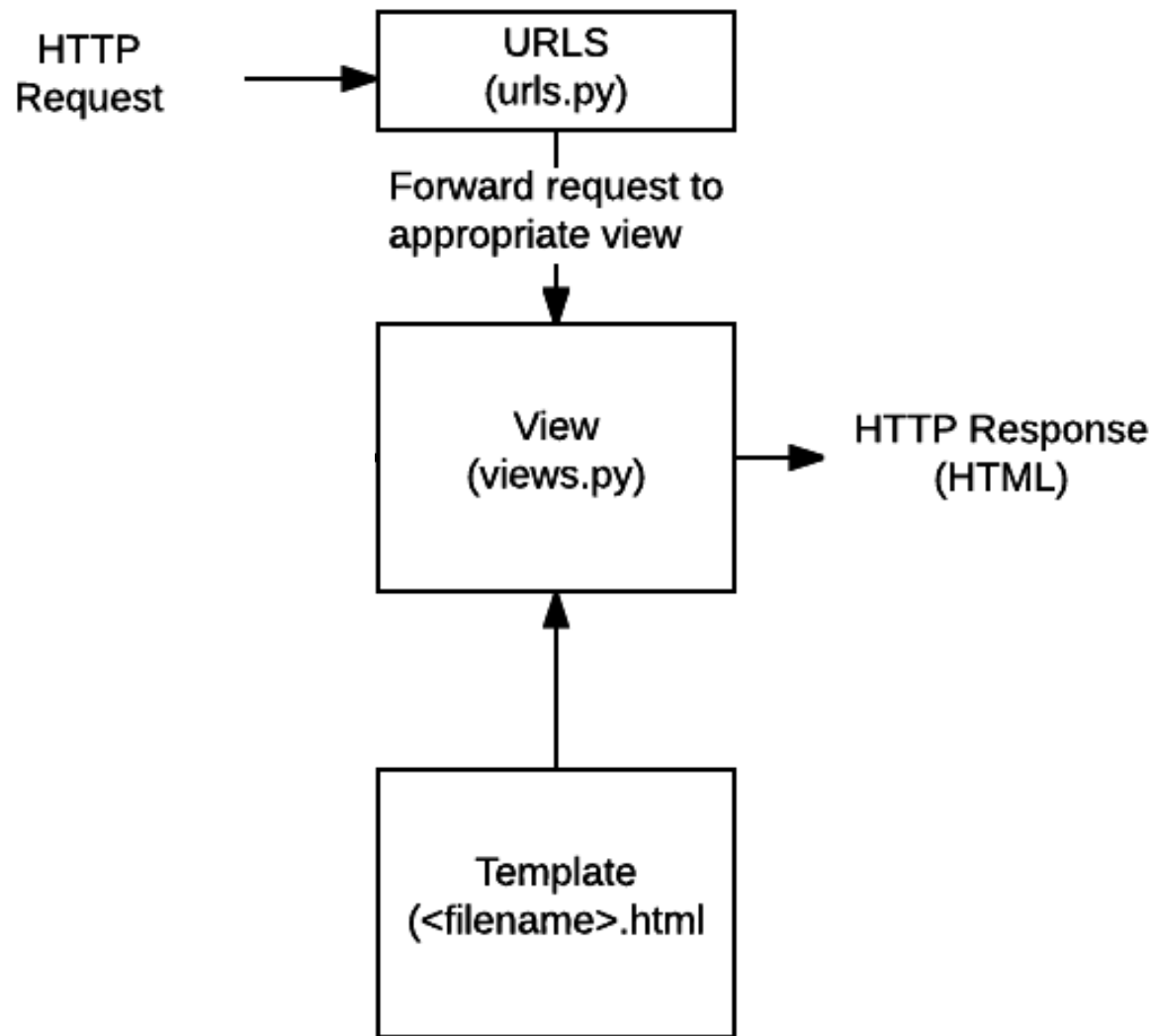
Boston University

Aaron Stevens

Learning Objectives

- Understand the two-step HTTP `GET/POST` cycle for processing HTML forms
- Trace how a request flows through Django's URL dispatcher, view, and template
- Write a view that renders a form on `GET` and processes submitted data on `POST`
- Explain why `{% csrf_token %}` is required in every `POST` form
- Access submitted form fields using the `request.POST` dictionary
- Apply the Post-Redirect-Get pattern to prevent duplicate form submissions

Elements of a Django Web Application



Connecting URLs to Views (urls.py)

Add these URL patterns to connect browser requests to view functions:

```
from django.urls import path
from . import views

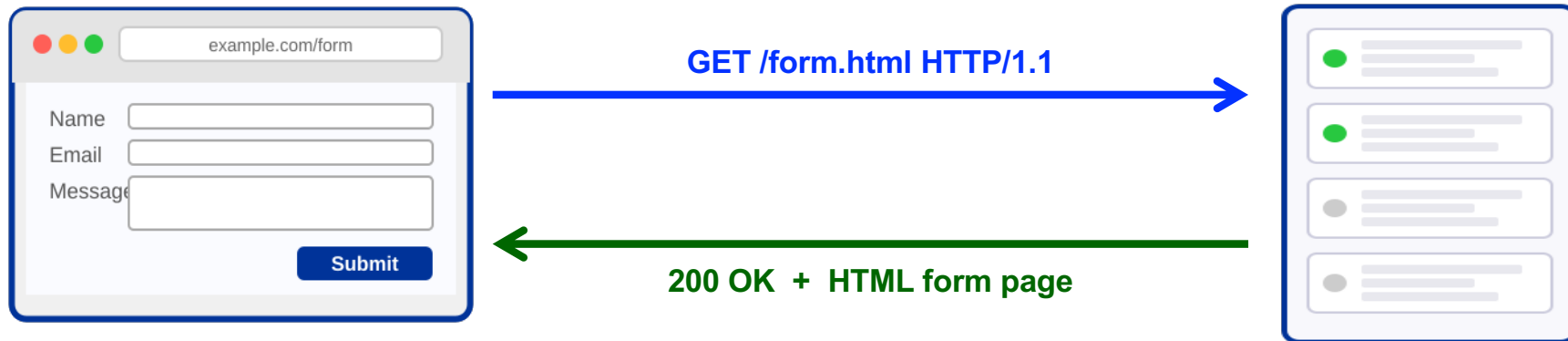
urlpatterns = [
    path('form/', views.show_form, name='show_form'),
    path('submit/', views.submit, name='submit'),
]
```

How Django routes a request:

- Browser requests `/form/` → Django calls `show_form(request)`
- Browser submits form to `/submit/` → Django calls `submit(request)`
- `name=` lets templates use `{% url 'show_form' %}` instead of hardcoding paths

STEP 1

HTTP GET — Requesting a Form



Client (Web Browser)

Server

HTTP Request

```
GET /form.html HTTP/1.1
Host: example.com
Accept: text/html
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html

<form method="post"> ... </form>
```

Note: GET requests have **no request body**. They only retrieve data from the server.

Key idea: The browser sends a GET request; the server responds with HTML containing a `<form method="post">` element. The browser renders the form — the same browser window that made the request.

STEP 1: HTTP GET to request a form; display in browser

```
def show_form(request):  
    '''Show the form to the user.'''  
  
    template_name = 'formdata/form.html'  
    return render(request, template_name)
```

form.html – Template

```
<!-- formdata/templates/formdata/form.html -->
{% extends 'formdata/base.html' %}

{% block content %}
<div>
  <h1>Contact Form</h1>

  <form action="{% url 'submit' %}" method="POST">
    {% csrf_token %}

    Name: <input type="text" name="name"><br>
    Favorite Color: <input type="text" name="favorite_color"><br>
    <input type="submit">

  </form>
</div>
```

Cross-Site Request Forgery (CSRF) Protection

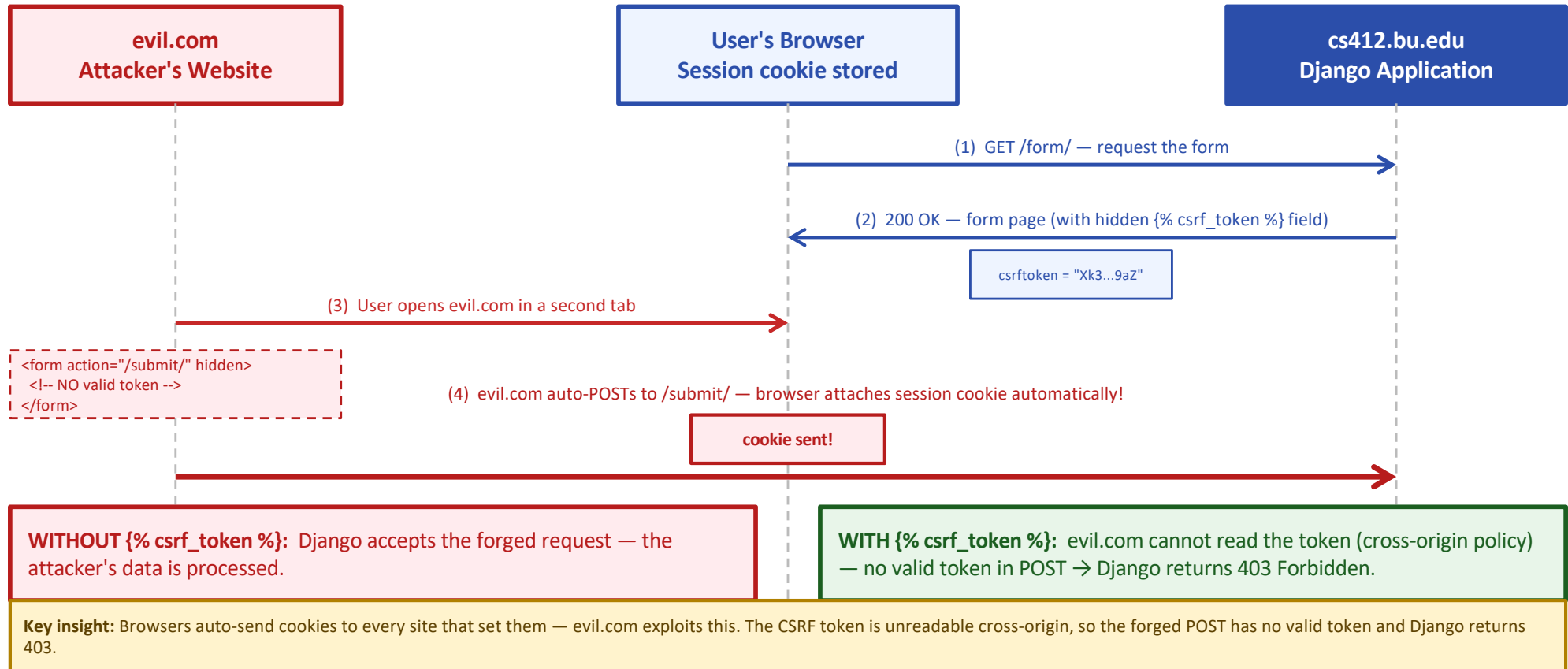
CSRF is an attack where a malicious site tricks your browser into submitting a form to a different site without your knowledge.

- Django defends against this by embedding a secret token in every POST form
- `{% csrf_token %}` adds a hidden `<input>` field containing a unique per-session token
- When the form is submitted, Django checks that the token is present and valid
- If the token is missing or wrong, Django returns a `403 Forbidden` error
- Place `{% csrf_token %}` scriptlet immediately inside every `<form method="post">` tag

 Omitting `{% csrf_token %}` is the #1 cause of mysterious 403 errors on form submissions.

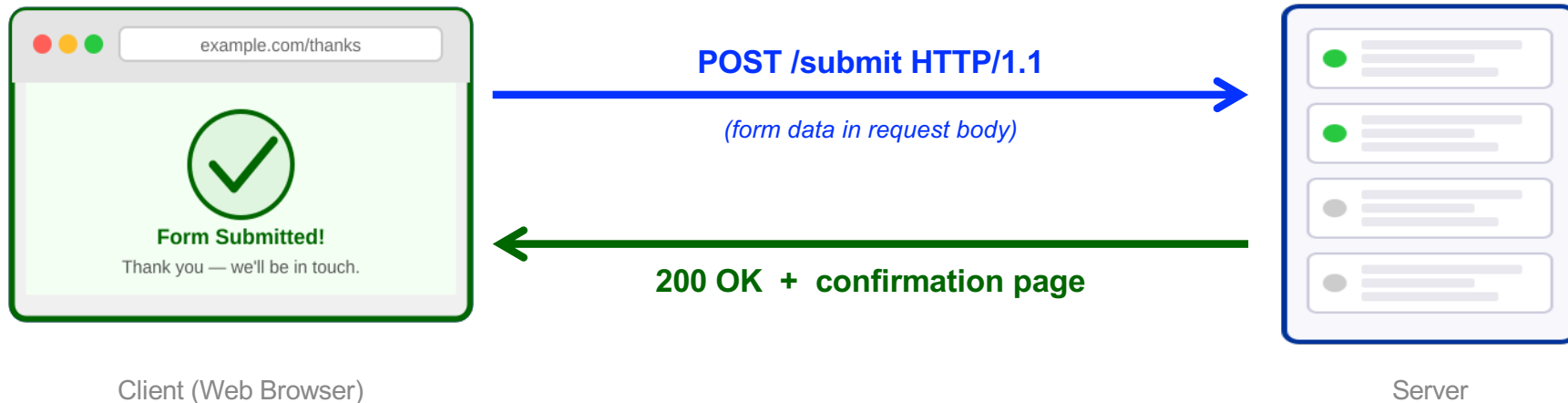
Cross-Site Request Forgery (CSRF) — How the Attack Works

The attacker's site tricks the user's browser into sending a forged request using the user's existing session.



STEP 2

HTTP POST — Submitting Form Data



HTTP Request

```
POST /submit HTTP/1.1
Host: example.com
Content-Type: application/
x-www-form-urlencoded

name=Alice+Smith
email=alice%40bu.edu
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Form submitted!</h1>
<p>Thank you, Alice.</p>
```

GET vs POST

GET: data in URL query string — visible, bookmarkable

POST: data in request body — hidden from URL, no size limit

Key idea: When the user clicks Submit, the browser sends a POST request. Form values travel in the **request body** — not the URL. The server processes them and responds; the same browser window displays the confirmation page.

Accessing Submitted Data: request.POST

request.POST is a dictionary-like object populated with the form field values sent in the HTTP POST body.

- Keys are the `name` attributes of `<input>` elements in your HTML form
- `request.POST['name']` retrieves the value of `<input name="name">`
- Use `request.POST.get('field', '')` for a safe read — returns `''` if the key is missing
- Use `if request.method == 'POST':` to check for a POST request (preferred)
- `request.POST` is empty for GET requests — only populated on POST

 `request.GET` works the same way for URL query string parameters (`?key=value`).

STEP 2: HTTP POST to submit/process a form

```
def submit(request):
    '''Process the form submission, and generate a result.'''

    template_name = "formdata/confirmation.html"
    print(request.POST)

    # check if POST data was sent with the HTTP POST message:
    if request.POST:

        # extract form fields into variables:
        name = request.POST['name']
        favorite_color = request.POST['favorite_color']

        # create context variables for use in the template
        context = {
            'name': name,
            'favorite_color': favorite_color,
        }

        # delegate the response to the template, provide context variables
        return render(request, template_name=template_name, context=context)

    # default behavior: handle the GET request
    template_name = 'formdata/form.html'
    return render(request, template_name)
```

The Post-Redirect-Get (PRG) Pattern

After successfully processing a POST request, redirect the browser to a new URL rather than rendering a response directly.

- Problem: if you render a page after POST, the browser can replay the POST on refresh
- A replayed POST creates duplicate submissions — orders placed twice, records inserted twice
- Solution: `return redirect('view_name')` — sends the browser to a new URL via GET
- The browser then fetches the confirmation page with GET — safe to refresh
- ```
from django.shortcuts import redirect
→ return redirect('confirmation')
```

 Add `path('confirmation/', views.confirmation, name='confirmation')` to `urls.py`.

## STEP 2: HTTP POST to submit/process a form

```
12 / examples / django / formdata / templates / formdata / confirmation.html / ...
1 <!-- formdata/templates/formdata/confirmation.html -->
2 {% extends 'formdata/base.html' %}
3
4 {% block content %}
5 <div>
6 <h1>Confirmation Page</h1>
7
8 <p>
9 Hi, {{name}}!
10 </p>
11
12 <p>
13 You said your favorite color was {{favorite_color}}.
14 </p>
15 </div>
16 {% endblock %}
```

# Summary

- HTML forms use two requests: `GET` (display the form) and `POST` (submit data)
- URL patterns in `urls.py` map paths to view functions using `path()`
- Views check `request.method` to handle `GET` vs `POST` in the same function
- Always include `{% csrf_token %}` inside `<form method="post">` — Django requires it
- Access submitted fields with `request.POST['field_name']` (use `.get()` to be safe)
- After processing a `POST`, redirect with `redirect('view_name')` — never render directly